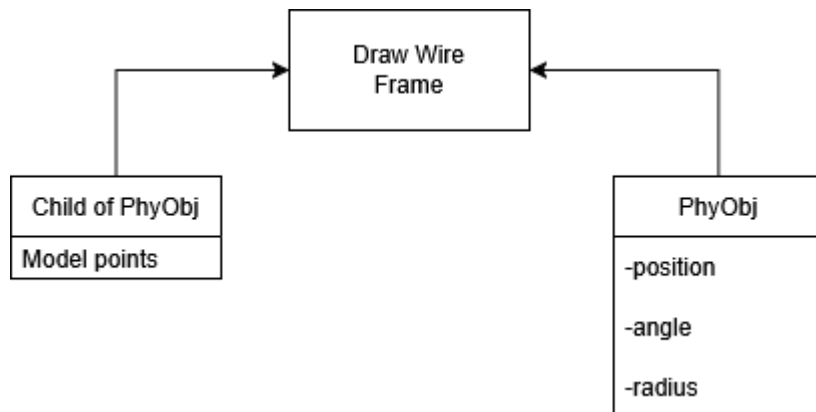


Structure

Every thing that can move in the game is derived from the physics object. Making map of char can enable you to change the terrain type latter.



Angle from the velocity vector

```
atan2f(-0.6,-0.8); //automatically gives the angle in the correct quadrant by determining the
```

```
float v_angle = atan2f(obj->velocity.y, obj->velocity.x);
```

Kinematics

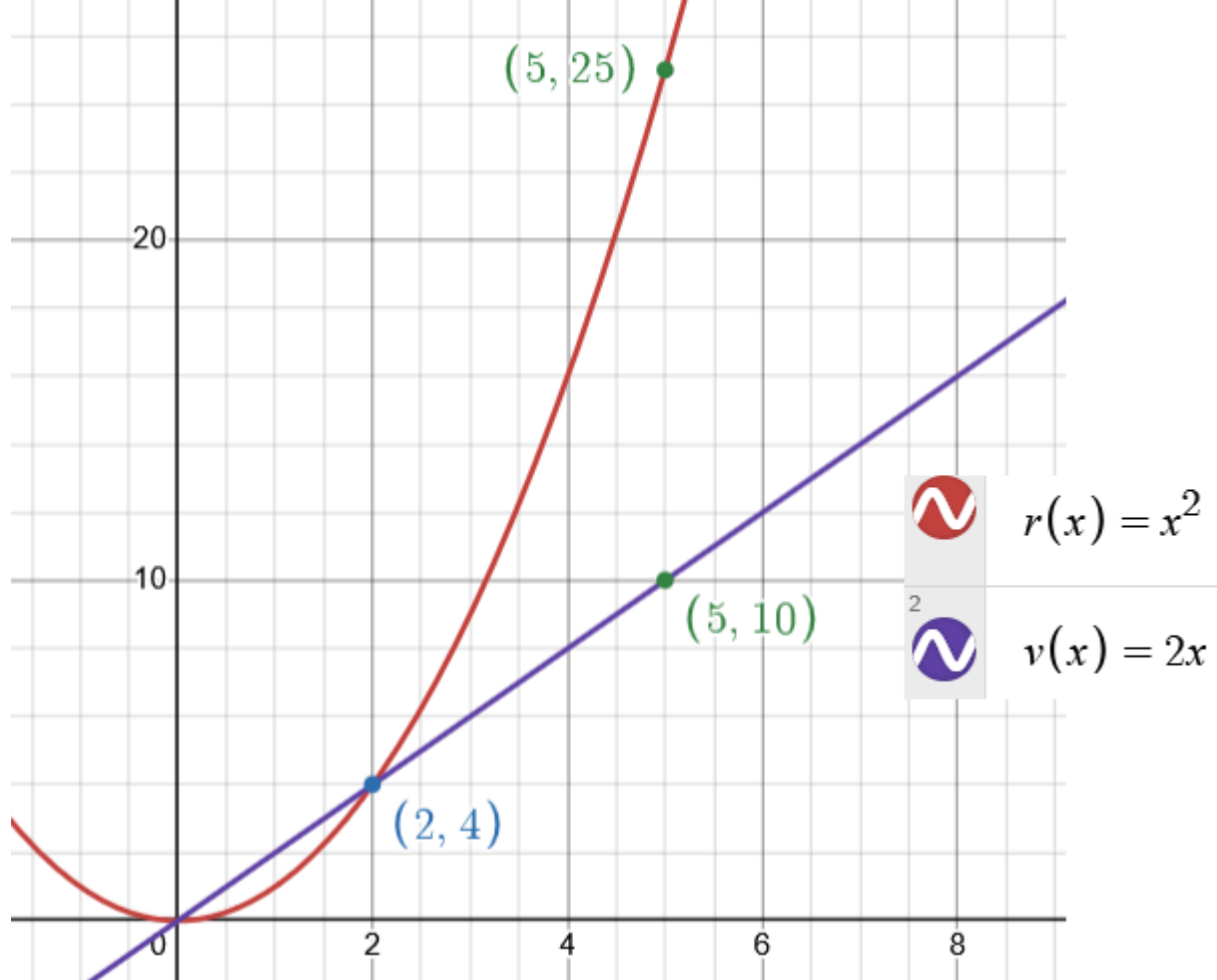
<https://www.youtube.com/watch?v=QgDR8LrRZhk&t=1946s>

From the **instantaneous** position $\mathbf{r} = \mathbf{r}(t)$, instantaneous meaning at an instant value of time t , the instantaneous velocity $\mathbf{v} = \mathbf{v}(t)$ and acceleration $\mathbf{a} = \mathbf{a}(t)$ have the general, coordinate-independent definitions;^[7]

$$\mathbf{v} = \frac{d\mathbf{r}}{dt}, \quad \mathbf{a} = \frac{d\mathbf{v}}{dt} = \frac{d^2\mathbf{r}}{dt^2}$$

[1]

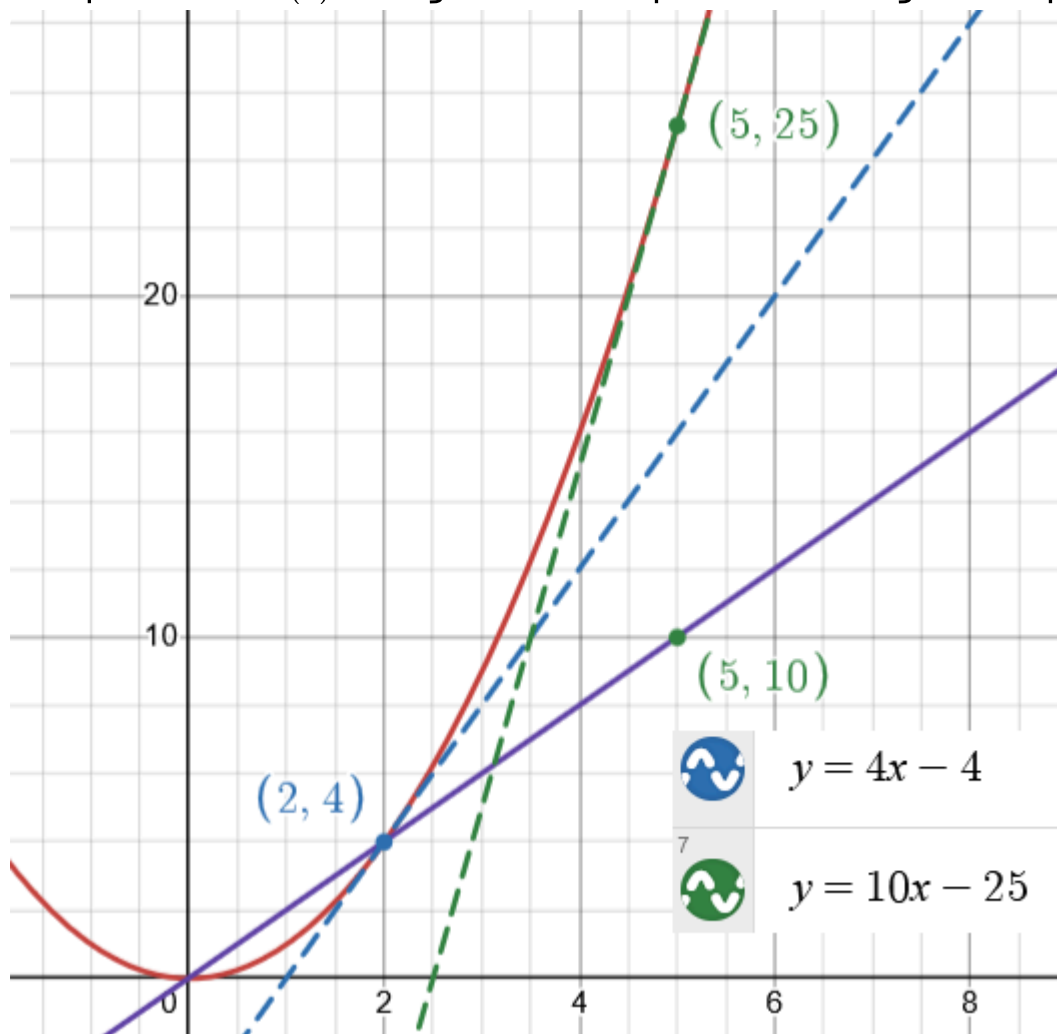
Consider the following velocity and position functions along with the highlighted points:



We can observe the relation:

$$\frac{d}{dx}r(x) = \frac{d}{dx}x^2 \rightarrow 2x = v(x)$$

The points on $v(x) = 2x$ give the slope of the tangent to points on $r(x) = x^2$:



We can also observe the anti derivate relation:

$$\int v(x) dx = \int 2x dx = 2 \frac{x^2}{2} + C \rightarrow x^2 + C = r(x)$$

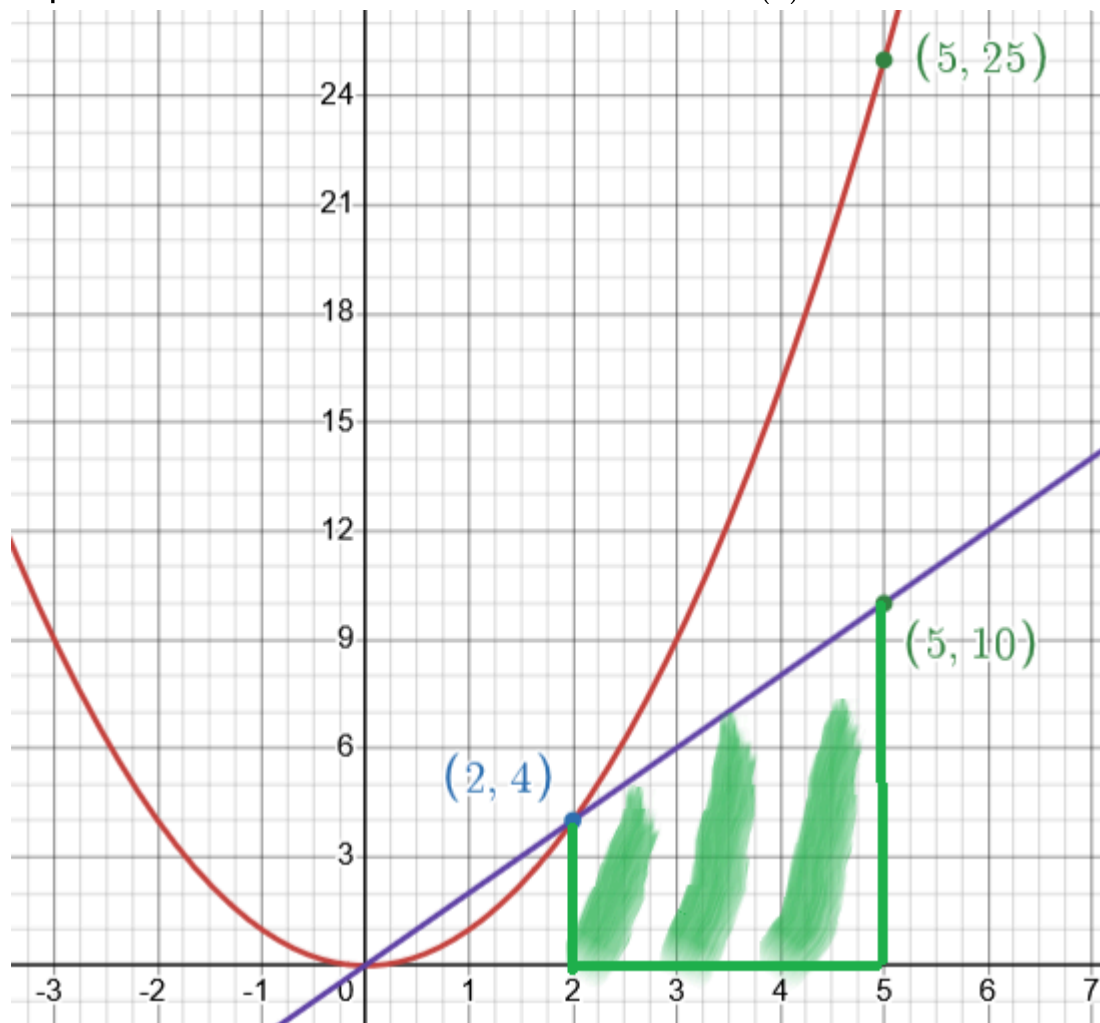
Notice how we get a family of position curves instead of a single position function cause the derivate of a constant is zero. So the information of the constant get's lost when we apply a derivate.

$$\int_2^5 v(x) dx$$

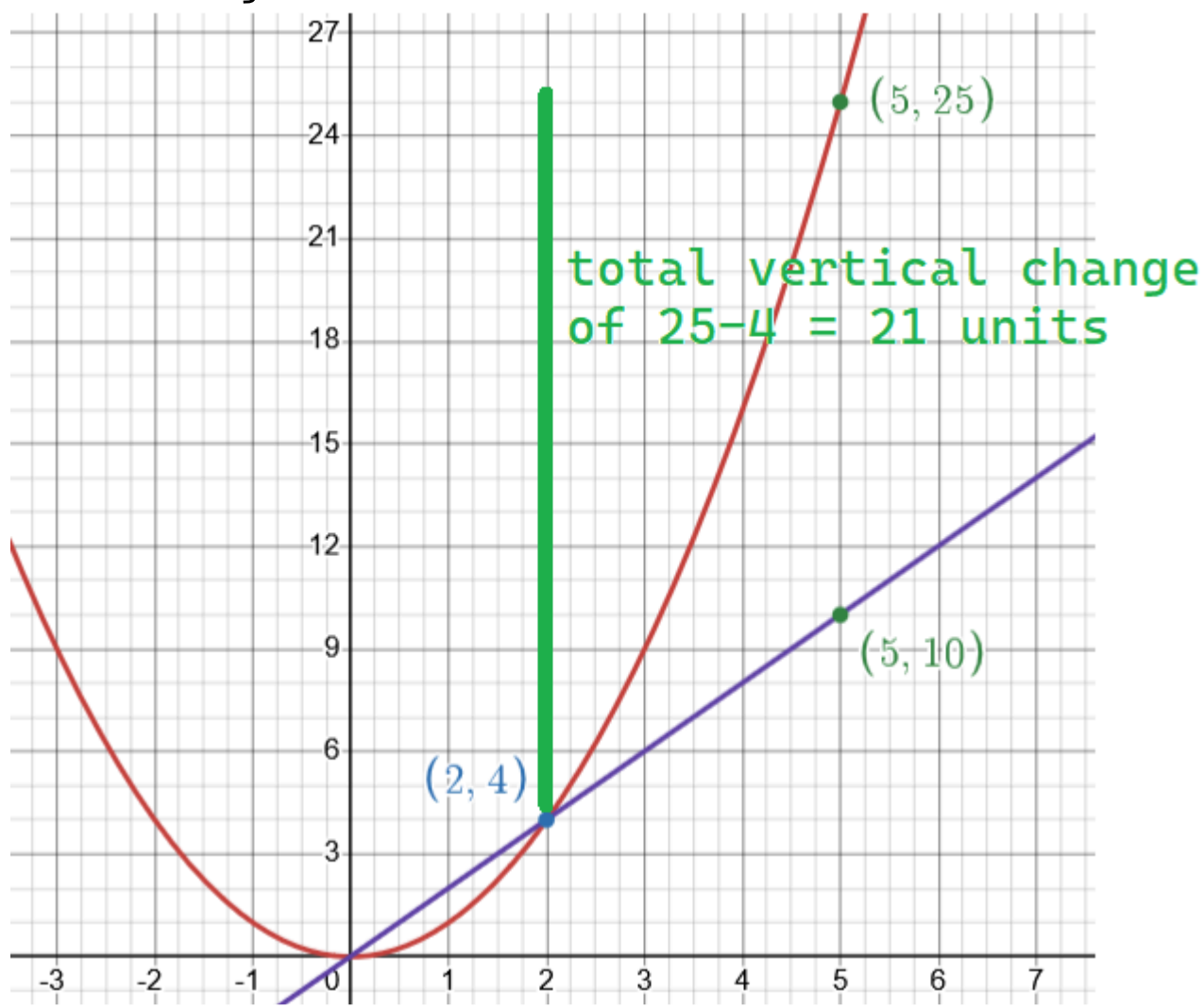
×

= 21

Represents the area under the curve of $v(x)$ from 2 to 5:



On the graph of $r(x)$ the definite integral from 2 to 5 represents the total vertical change:



Equations of Motion

These equations are a direct consequence of the relation:

From the **instantaneous** position $\mathbf{r} = \mathbf{r}(t)$, instantaneous meaning at an instant value of time t , the instantaneous velocity $\mathbf{v} = \mathbf{v}(t)$ and acceleration $\mathbf{a} = \mathbf{a}(t)$ have the general, coordinate-independent definitions;^[7]

$$\mathbf{v} = \frac{d\mathbf{r}}{dt}, \quad \mathbf{a} = \frac{d\mathbf{v}}{dt} = \frac{d^2\mathbf{r}}{dt^2}$$

Constant translational acceleration in a straight line [\[edit \]](#)

These equations apply to a particle moving linearly, in three dimensions in a straight line with constant **acceleration**.^[9] Since the position, velocity, and acceleration are collinear (parallel, and lie on the same line) – only the magnitudes of these vectors are necessary, and because the motion is along a straight line, the problem effectively reduces from three dimensions to one.

$$v = v_0 + at \quad [1]$$

$$r = r_0 + v_0 t + \frac{1}{2}at^2 \quad [2]$$

$$r = r_0 + \frac{1}{2}(v + v_0)t \quad [3]$$

$$v^2 = v_0^2 + 2a(r - r_0) \quad [4]$$

$$r = r_0 + vt - \frac{1}{2}at^2 \quad [5]$$

where:

- r_0 is the particle's initial **position**
- r is the particle's final position
- v_0 is the particle's initial **velocity**
- v is the particle's final velocity
- a is the particle's **acceleration**
- t is the **time interval**

Equations [1] and [2] are from integrating the definitions of velocity and acceleration,^[9] subject to the initial conditions $\mathbf{r}(t_0) = \mathbf{r}_0$ and $\mathbf{v}(t_0) = \mathbf{v}_0$;

$$\mathbf{v} = \int \mathbf{a} dt = \mathbf{a}t + \mathbf{v}_0, \quad [1]$$

$$\mathbf{r} = \int (\mathbf{a}t + \mathbf{v}_0) dt = \frac{\mathbf{a}t^2}{2} + \mathbf{v}_0 t + \mathbf{r}_0, \quad [2]$$

in magnitudes,

$$v = at + v_0, \quad [1]$$

$$r = \frac{at^2}{2} + v_0 t + r_0. \quad [2]$$

You can also derive 1 and 2 if you consider two points on $r(x)$:

$$\frac{r_2 - r_1}{t} = v$$

$$r_2 = vt + r_1$$

where r_1 is the initial position and r_2 is the final position. In simple words:

we can obtain the final position of an object given that we know:

1. the velocity of the interval
2. time of the interval
3. initial position of the object

Same procedure can be applied on velocity to obtain:

$$\frac{v_2 - v_1}{t} = a$$

$$v_2 = at + v_1$$

In practice t would be the time interval between successive frame

Delta Time in Physics Calculations

<https://youtu.be/QgDR8LrRZhk?t=349>

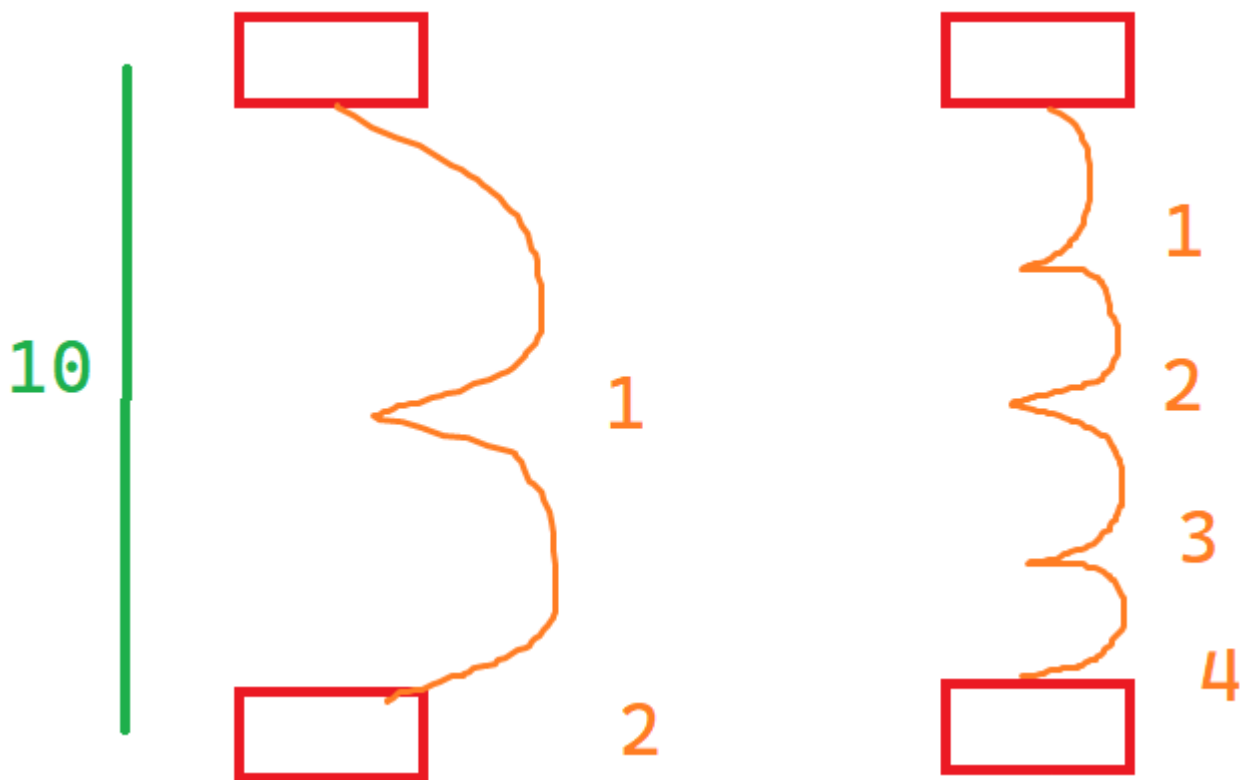
How would you make it so each object on the screen moves the same distance for 1 second regardless of the number of frames drawn per second:

```
// apply forces to acceleration
objects_list[i]->acceleration.y += 2.0f;

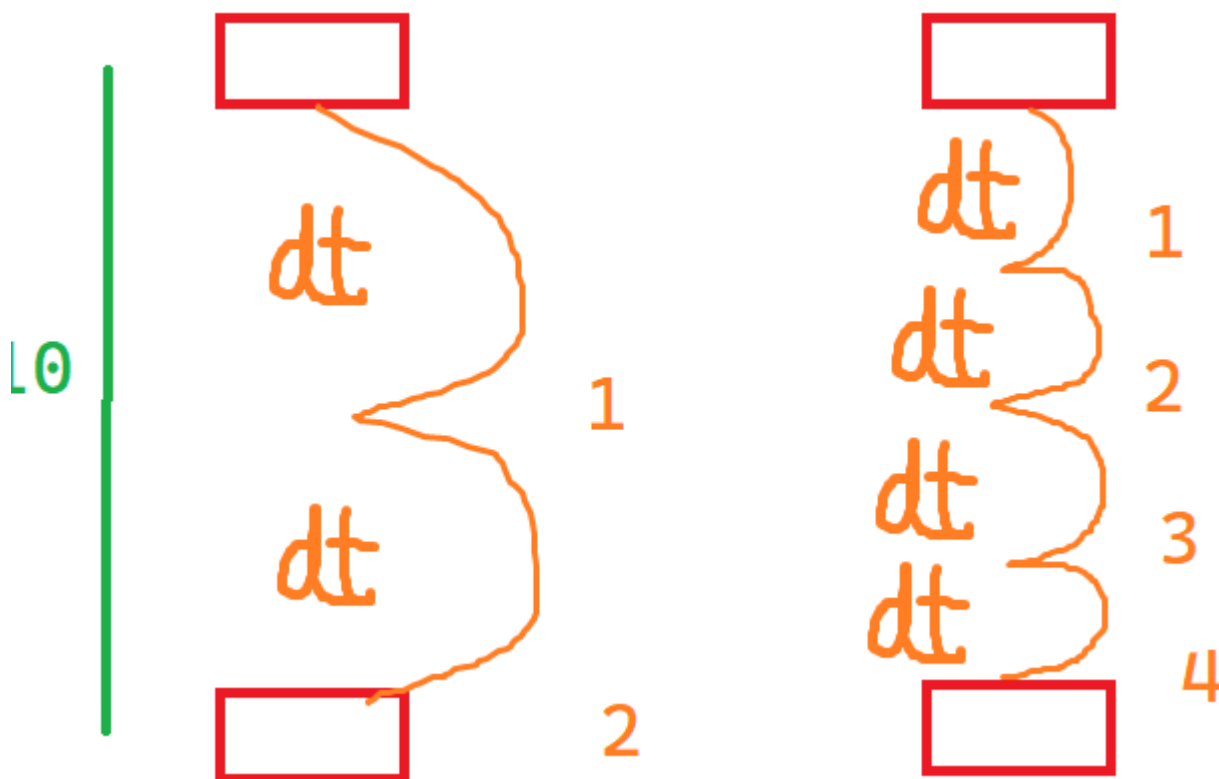
objects_list[i]->velocity.x += objects_list[i]->acceleration.x* GetFrameTime();
objects_list[i]->velocity.y += objects_list[i]->acceleration.y* GetFrameTime();

objects_list[i]->position.x += objects_list[i]->velocity.x * GetFrameTime();
objects_list[i]->position.y += objects_list[i]->velocity.y * GetFrameTime();
```

`GetFrameTime() / dt` makes sure that if the current frame took too long to render then the object would be drawn accordingly. A high `dt` makes sure that we catch up to the physics simulation.



We are essentially dividing our journey into equal steps that get taken at each `dt`. The larger the fps the smaller the `dt` and the smaller the fps the larger the `dt`:



If we get rid of the `dt`:

```
// apply forces to acceleration
objects_list[i]->acceleration.y += 2.0f;

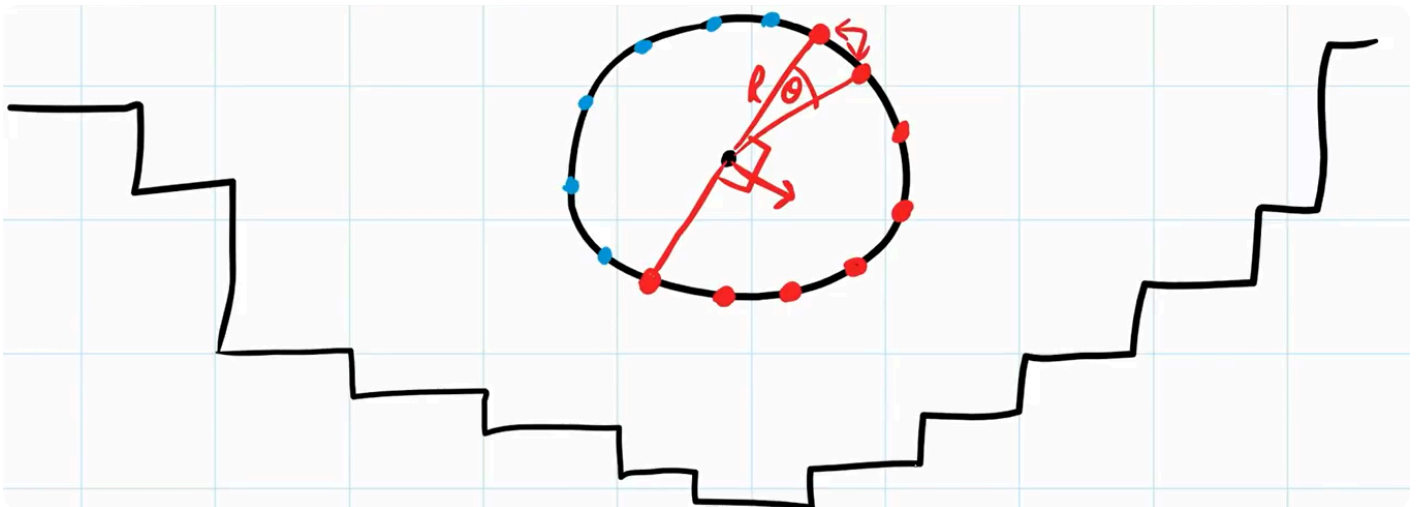
objects_list[i]->velocity.x += objects_list[i]->acceleration.x;
objects_list[i]->velocity.y += objects_list[i]->acceleration.y;
```

$dt=1$ 

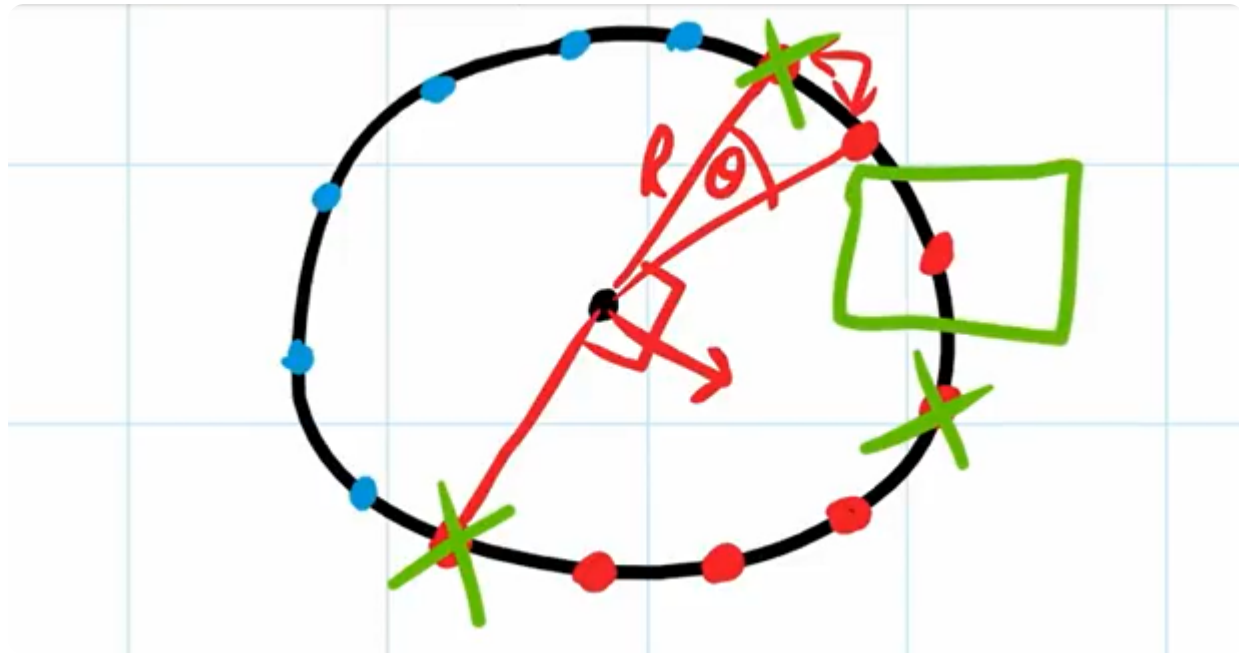
fps

Collision Detection

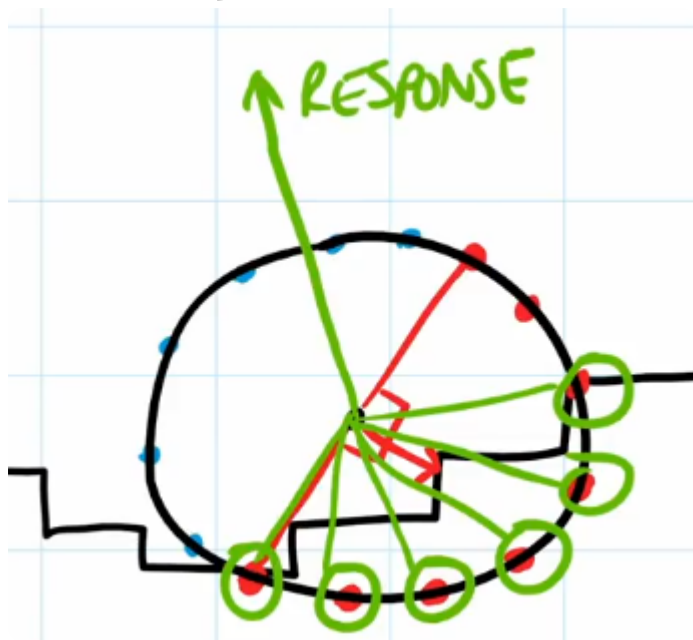
- Ensure that the pixel difference between each point in the circle is less than 1 map pixel:



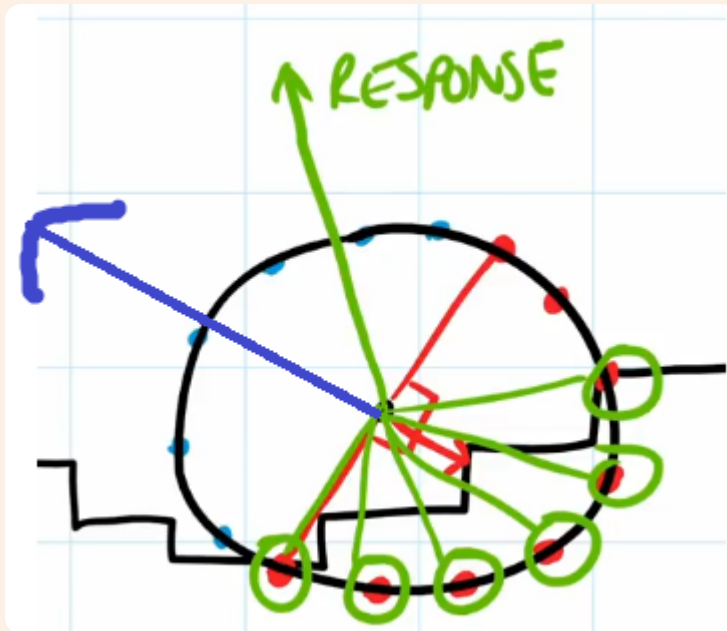
Or else collision is not guaranteed with at least 1 map pixel.



"Response approximates the direction in which then object need to move to escape being in collision"^[3]:



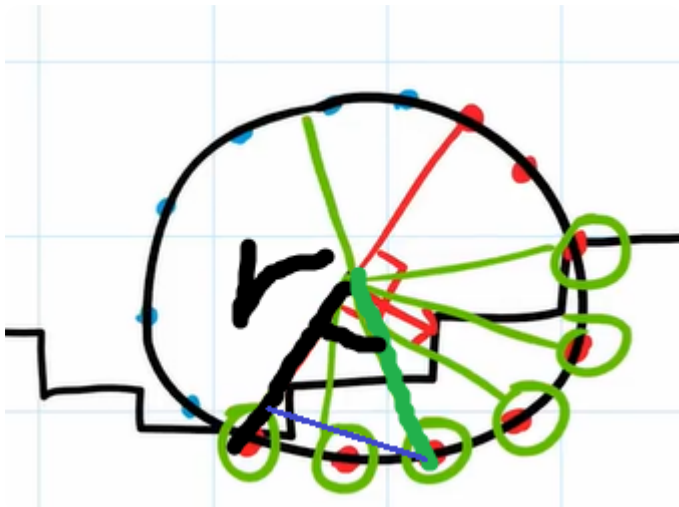
Ⓜ Isn't a better response vector something like:



Which would be easy to compute than doing the vector addition and inverting the result ? invert of the velocity vector. No since it would not account for the collision cases

<https://youtu.be/EH7aJvQpW3U?t=1443>

So deriving the points:



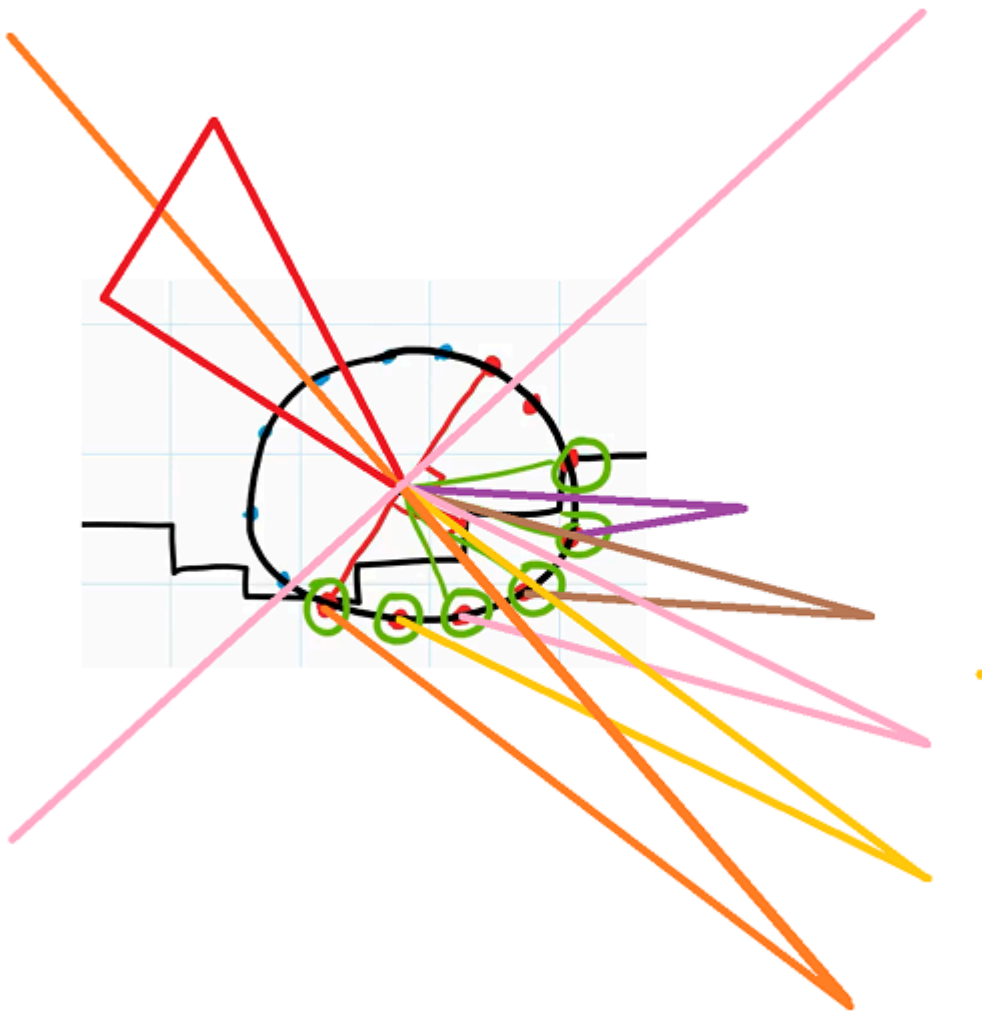
```
// Calculate test point on circumference of circle
float fTestPosX = (p->radius) * cosf(r) + fPotentialX;
float fTestPosY = (p->radius) * sinf(r) + fPotentialY;
```

```
// Calculate test point on circumference of circle  
float fTestPosX = (p->radius) * cosf(r) + fPotentialX;  
float fTestPosY = (p->radius) * sinf(r) + fPotentialY;
```

this part actually
calculates the (x,y)
of the point

this just
translate the
point according to
the position on
the map

Now notice how translating the points puts them on the real map in-game.
Hence we can use these points to access the map at the said location.



Reflection Vector

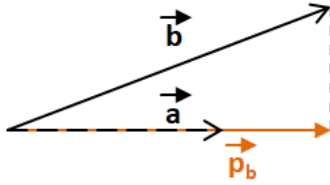
DotProduct

The most essential mathematic tool that is going to be used is the dot product of two vectors, so at first a short recapitulation.

The dot product of two vectors $\mathbf{a}$ and $\mathbf{b}$ is calculated as $\mathbf{a} \cdot \mathbf{b} = a_1 * b_1 + a_2 * b_2 \dots + a_n * b_n$.

Generally, the important property of the dot product is that it can be used to calculate the angle α between two vectors (because $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| * |\mathbf{b}| * \cos(\alpha)$).

However, another geometric property is most often even more useful:

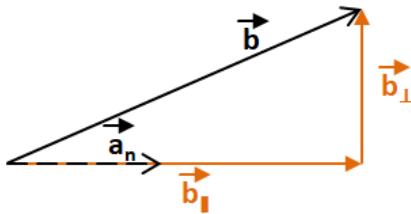


As shown in the figure above, the vector $\mathbf{b}$ is projected onto vector $\mathbf{a}$, resulting in $\mathbf{p_b}$. The dot product can be used to calculate the length of the projected vector $\mathbf{p_b}$, i.e. $|\mathbf{p_b}|}$.

The concrete relation is as follows: The dot product is equal to the signed length of the projection of $\mathbf{b}$ onto $\mathbf{a}$ multiplied by the length of $\mathbf{a}$. So $|\mathbf{p_b}| = (\mathbf{a} \cdot \mathbf{b} / |\mathbf{a}|)$.

If $\mathbf{a}$ is a unit vector, then the dot product provides the desired result directly without division: $|\mathbf{p_b}| = \mathbf{a_n} \cdot \mathbf{b}$.

This fact provides an even more practical tool which is shown in the following figure:



It can be used to split $\mathbf{b}$ into two vectors: One vector that is parallel to $\mathbf{a}$ which is denoted as $\mathbf{b_{||}}$, the other vector that is perpendicular to $\mathbf{a}$ which is denoted as $\mathbf{b_{\perp}}$.

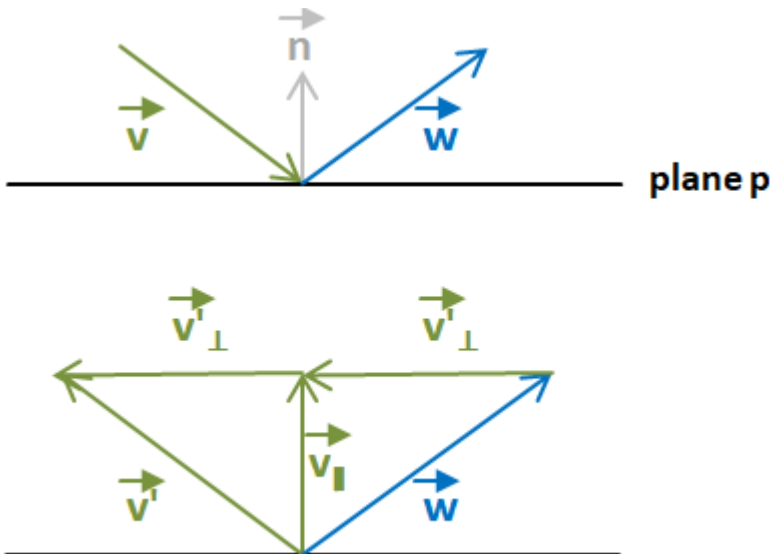
The length of $\mathbf{b_{||}}$ is: $|\mathbf{b_{||}}| = \mathbf{a_n} \cdot \mathbf{b}$.

So to get the actual vector $\mathbf{b_{||}}$, just multiply it with $\mathbf{a_n}$: $\mathbf{b_{||}} = (\mathbf{a_n} \cdot \mathbf{b}) * \mathbf{a_n}$.

Finally, getting $\mathbf{b_{\perp}}$ is straight-forward because $\mathbf{b}$ is the sum of $\mathbf{b_{||}}$ and $\mathbf{b_{\perp}}$: $\mathbf{b} = \mathbf{b_{\perp}} + \mathbf{b_{||}}$.

Thus $\mathbf{b_{\perp}} = \mathbf{b} - \mathbf{b_{||}} = \mathbf{b} - (\mathbf{a_n} \cdot \mathbf{b}) * \mathbf{a_n}$.

<https://www.sunshine2k.de/articles/coding/vectorreflection/vectorreflection.html>



Implementing the equation:

$$\vec{w} = \vec{v}_{||} - \vec{v'_{\perp}}$$

instead of:

$$\mathbf{w} = \mathbf{v} - 2 * (\mathbf{v} \cdot \mathbf{n}) * \mathbf{n}$$

adding about 21 `phy_obj` make the physics loop take about 500+ microseconds in both implementation. So the difference both approaches is negligible,

in the following code:

```
if (collision_occured == true)
{

    // invert the response vector

    response.x *= -1;

    response.y *= -1;

    objects_list[i]->is_stable = true;

    // calculate normal collision plane

    float response_angle = atan2f(response.y, response.x);

    float normal_angle = response_angle + (PI / 2.0); //+ or minus does not m

    // normalize the response vector

    float response_mag = sqrtf((response.x * response.x) + (response.y * resp

    response.x = response.x / response_mag;

    response.y = response.y / response_mag;

    // invert the velocity vector for reflection calculations

    objects_list[i]->velocity.x *= -1;

    objects_list[i]->velocity.y *= -1;
```

```

        // projection of velocity to n i.e v_parallel calculation

        // magnitude of v_parallel

        float dot = (objects_list[i]->velocity.x * response.x) + (objects_list[i]

        // actual v_parallel calculation

        Vector2 v_parallel{response.x * dot, response.y * dot};

        // v_perpendicular calculation

        // v - v_parallel

        Vector2 v_perpendicular;

        v_perpendicular.x = objects_list[i]->velocity.x - v_parallel.x;

        v_perpendicular.y = objects_list[i]->velocity.y - v_parallel.y;

        // invert the v_parallel

        v_perpendicular.x *= -1;

        v_perpendicular.y *= -1;

        // final velocity reflection calculation

        Vector2 v_reflection;

        v_reflection.x = v_parallel.x + v_perpendicular.x;

        v_reflection.y = v_parallel.y + v_perpendicular.y;

        objects_list[i]->velocity.x = v_reflection.x;

        objects_list[i]->velocity.y = v_reflection.y;

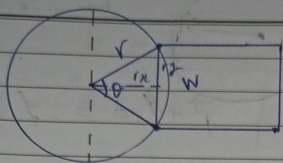
    }

```

Angle Step

How do we make sure that every physics object no matter the radius is able to detect collision with any map tile?

Given r, w find θ



$$\cos\left(\frac{\theta}{2}\right) = \frac{r \cos\left(\frac{\theta}{2}\right)}{r}$$

$$w = 2r \cos\left(\frac{\theta}{2}\right) \quad r \cos\left(\frac{\theta}{2}\right) = \frac{w}{2}$$

$$\sin\left(\frac{\theta}{2}\right) = \frac{r \sin\left(\frac{\theta}{2}\right)}{r}$$

$$\theta = \frac{\theta}{2} + \frac{\theta}{2}$$

$$\sin(\theta) = \sin\left(\frac{\theta}{2} + \frac{\theta}{2}\right)$$

$$\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$$

$$\sin(\alpha + \alpha) = \sin \alpha \cos \alpha + \cos \alpha \sin \alpha$$

$$\sin(2\alpha) = 2 \sin \alpha \cos \alpha$$

$$\sin\left(\frac{\theta}{2} + \frac{\theta}{2}\right) = \sin\left(\frac{\theta}{2}\right) \cos\left(\frac{\theta}{2}\right) + \cos\left(\frac{\theta}{2}\right) \sin\left(\frac{\theta}{2}\right)$$

$$\sin \theta = 2 \sin\left(\frac{\theta}{2}\right) \cos\left(\frac{\theta}{2}\right)$$

$$\sin \theta = 2 \frac{r \cos\left(\frac{\theta}{2}\right)}{r} \times \frac{r \sin\left(\frac{\theta}{2}\right)}{r}$$

$$\sin \theta = \frac{2ryx}{r^2}$$

$$r^2 = rx^2 + ry^2$$

$$rx^2 = r^2 - ry^2$$

$$rx = \sqrt{r^2 - ry^2}$$

$$\sin \theta = \frac{2ry \times \sqrt{r^2 - ry^2}}{r^2}$$

$$\sin \theta = \frac{x \cdot w}{x} \times \frac{\sqrt{r^2 - (\frac{w}{2})^2}}{r^2}$$

$$= \frac{w \times \sqrt{r^2 - \frac{w^2}{4}}}{r^2}$$

$$= \frac{w \times \sqrt{4r^2 - w^2}}{4r^2}$$

$$= \frac{w \times \sqrt{4r^2 - w^2}}{2} \times \frac{1}{r^2}$$

$$\cancel{r^2} \times \frac{1}{\cancel{r^2}}$$

$$(2r)^2 - (w)^2 = (2r+w)(2r-w)$$

$$\Rightarrow \frac{w \sqrt{4r^2 - w^2}}{2r^2} = \sin \theta$$

$$\theta = \sin^{-1} \left(\frac{w \sqrt{4r^2 - w^2}}{2r^2} \right)$$

$$\sin^{-1} \left(\frac{w \sqrt{(2r+w) \cdot (2r-w)}}{2r^2} \right)$$

Debris Rectangle

```

void debris::draw()

{

    //the center of the debris's phy_obj would be meeting point of the diagonals

    //of the rectangle of debris

    Rectangle rec;

    //top left corner

    rec.x = radius*cosf((3*PI)/4)+position.x;

    rec.y = radius*sinf((3*PI)/4)+position.y;

    //width of the rectangle = top right corner x - top left corner x
  
```



```

rec.width = (radius*cosf(PI/4)+position.x)-rec.x;

//height of the rectangle = top left corner y - bottom left corner y

rec.height = rec.y-(radius*sinf((5*PI)/4)+position.y);

//without this the rectangle was being drawn from the bottom left corner

//cause of the inverted origin?

rec.y -= rec.height;

//draw the rec

DrawRectangleRec(rec,DARKGREEN);

}

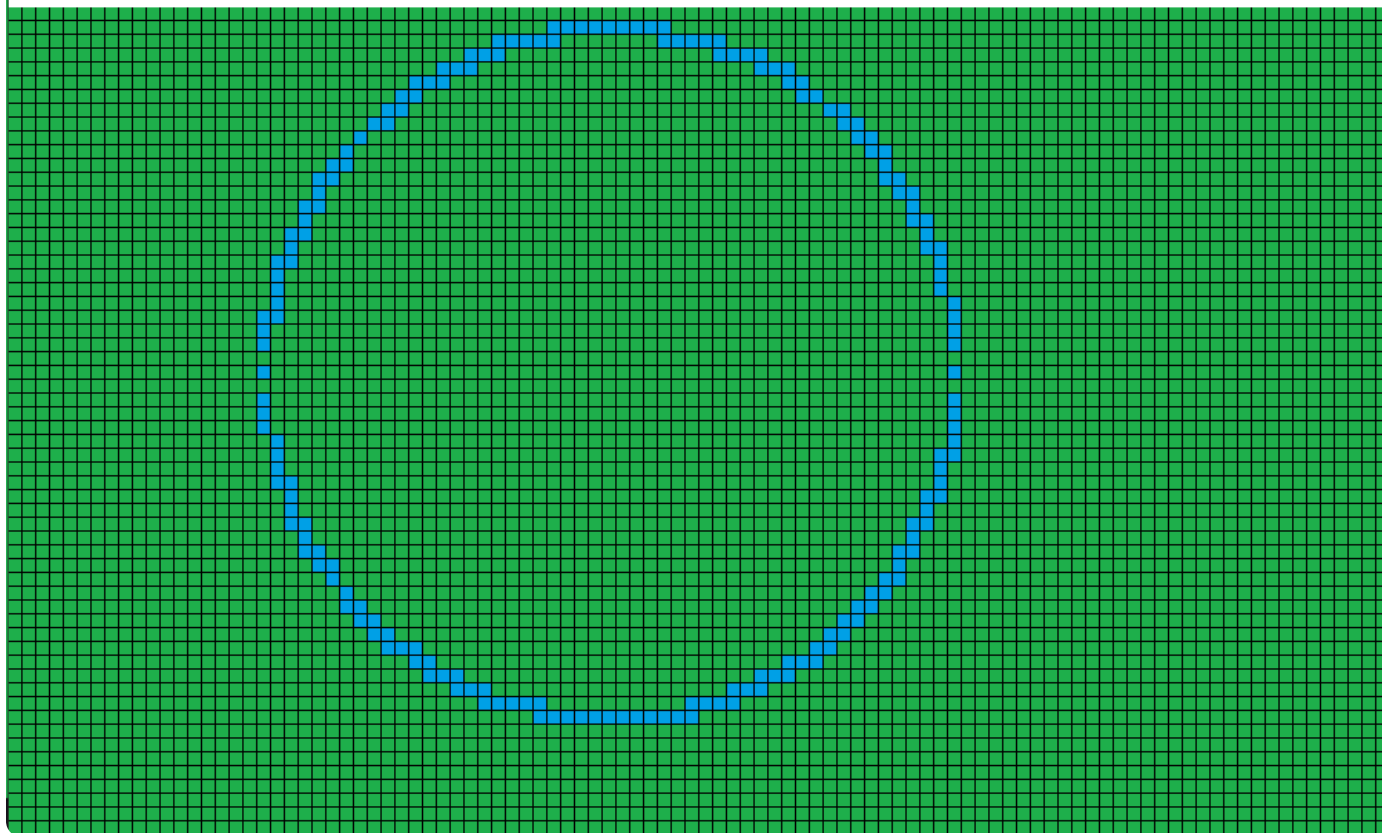
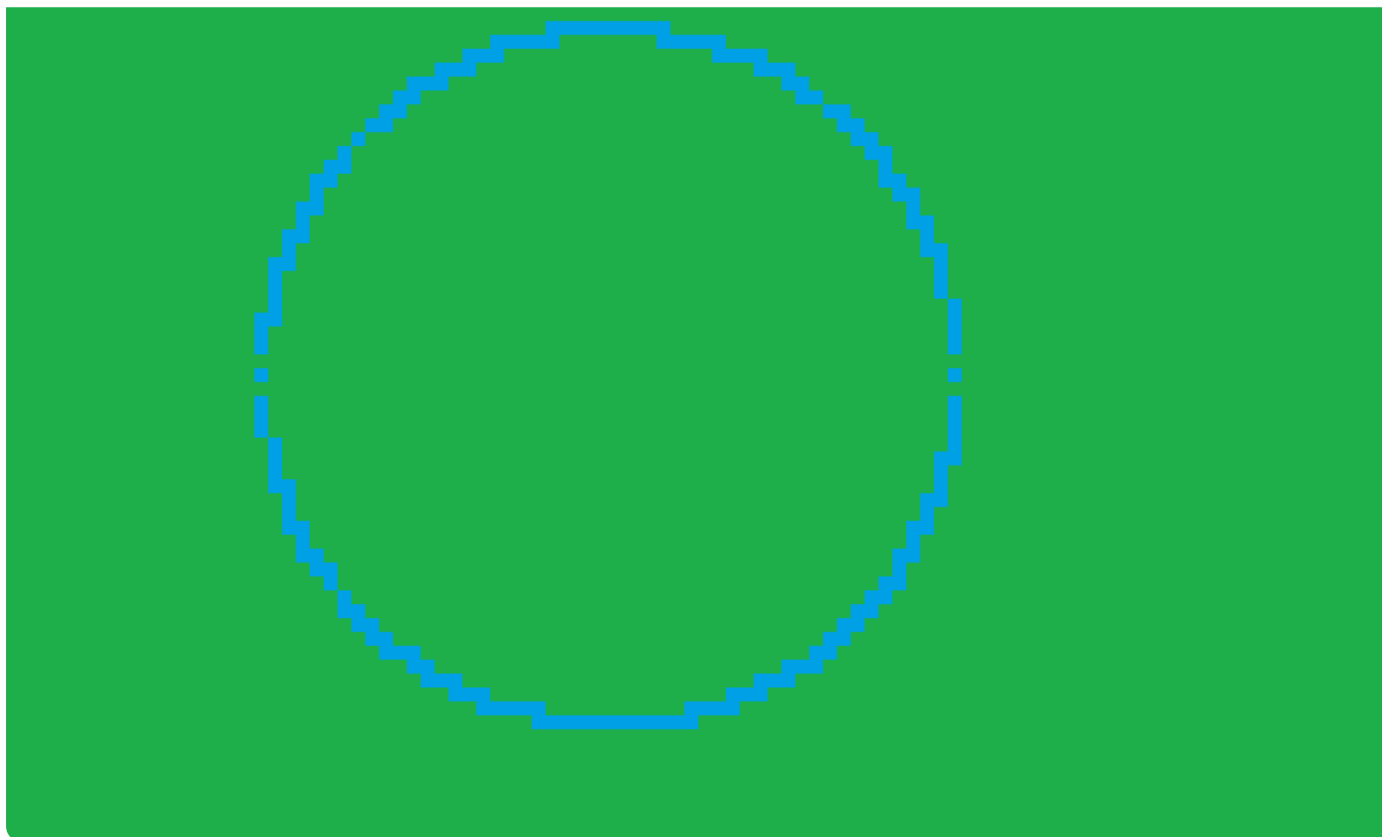
```

`radius*cosf((3*PI)/4)+position.x` cause each corner of the rectangle makes 90° angle so $\frac{90^\circ}{2} = \frac{\pi}{4}$

Mid Point Circle algorithm

The discontinuity discussed here

<https://web.archive.org/web/20120422045142/https://banu.com/blog/7/drawing-circles/> as x approaches the x-axis can be observed below:



Here is the discontinuity in action via the raylib pixel set function:

worms yunits clone



```
Vector2 mouse_pos = {650,350};

int radius = 340;
int map_x = mouse_pos.x / 10;
int map_y1 = 0;
int map_y2 = 0;

for (int x = radius * -1; x <= radius; x++)
{
    int y1 = sqrtf((radius * radius) - (x * x));
    int y2 = y1 * -1;

    //transform the pixel coordinates according to the mouse pos
    int xf = x + mouse_pos.x;
    int y1f = y1 + mouse_pos.y;
    int y2f = y2 + mouse_pos.y;

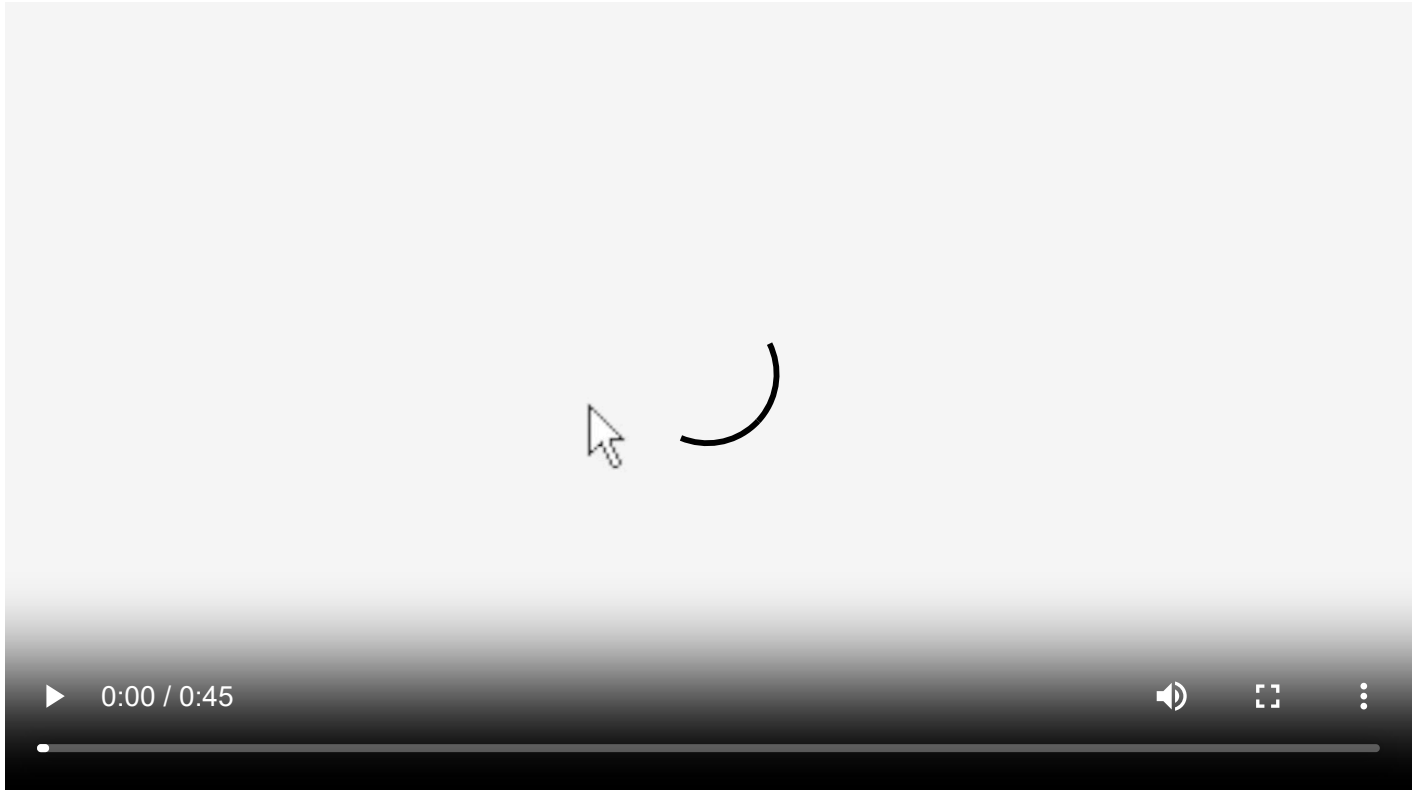
    // convert the pixel coordinates to map_coordinates
    //map_x = xf / 10;
    //map_y1 = y1f / 10;
    //map_y2 = y2f / 10;

    DrawPixel(xf,y1f,PINK);
    DrawPixel(xf,y2f,PINK);

    //mapp.map_grid[map_y1][map_x] = 'S';
    //mapp.map_grid[map_y2][map_x] = 'S';
}
```

The above code has a problem in that it draws the circle two times. In the video below the:

- The right circle is `DrawPixel(map_x+90,map_y1,BLUE);`
- The left circle is `DrawPixel(map_x,map_y2,BLACK);`



Hence we only need 1 `DrawPixel();` call because the outer loop for y covers the other half of the semicircle as well.

Filled circle

Filled circle is even easier than drawing the boundary of the circle since we just have to check the implicit equation $x^2 + y^2 = r^2$



```
Vector2 mouse_pos = {650, 350};

int radius = 100;

int map_x = mouse_pos.x / 10;

int map_y1 = 0;

int map_y2 = 0;

for (int y = radius * -1; y <= radius; y++)
{
    for (int x = radius * -1; x <= radius; x++)
    {
        // x^2+y^2 = r^2 check

        if ((x * x) + (y * y) <= (radius * radius))
        {

            int xf = x + mouse_pos.x;

            int y1f = y + mouse_pos.y;
```

```

        int y2f = (y * -1) + mouse_pos.y;

        DrawPixel(xf, y1f, BLUE);

        DrawPixel(xf, y2f, BLUE);

    }

}

}

```

? Question

Why couldn't we use the filled circle code to only draw the circle like so:



```

for (int y = radius * -1; y <= radius; y++)

{

    for (int x = radius * -1; x <= radius; x++)

    {

        // x^2+y^2 = r^2 check

        if ((x * x) + (y * y) == (radius * radius))

        {

            int xf = x + mouse_pos.x;

            int y1f = y + mouse_pos.y;

            int y2f = (y * -1) + mouse_pos.y;

            DrawPixel(xf, y1f, BLUE);

```

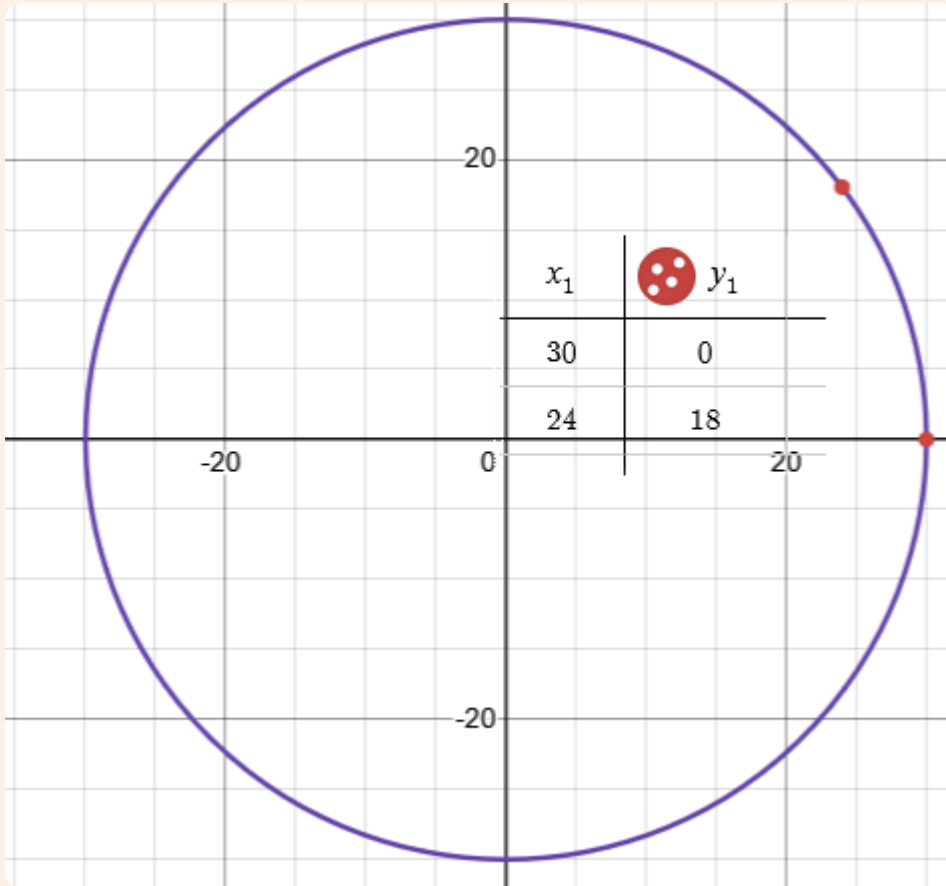
```
DrawPixel(xf, y2f, BLUE);
```

```
}
```

```
}
```

```
}
```

Because it's only gonna work where $x^2 + y^2$ gets satisfied exactly:



Explosion Function

```
void explosion(map *mapp, phy_engine *engine)
{
    Vector2 mouse_pos = GetMousePosition();

    int radius = 100; // radius in pixel values not in map coordinates

    float map_x = mouse_pos.x / 10;

    float map_y = 0;

    for (int y = radius * -1; y <= radius; y++)
    {
```

```

for (int x = radius * -1; x <= radius; x++)

{

    // x^2+y^2 = r^2 check equation of circle

    if ((x * x) + (y * y) <= (radius * radius))

    {

        // transform the pixel coordinates according to the mouse pos

        // and then convert the pixel coordinates to map_coordinates

        map_x = (x + mouse_pos.x) / 10;

        map_y = (y + mouse_pos.y) / 10;

        mapp->change_grid({map_x, map_y}, 'S');

    }

}

}

```

```

Node<phy_obj *> *tmp = engine->head;

```

```

Vector2 blast_velocity{};

```

```

while (tmp != nullptr)

```

```

{

```

```

    phy_obj *obj = tmp->data;

```

```

    // vector from the explosion position to the phy_obj position

```

```

    blast_velocity.x = obj->position.x - mouse_pos.x;

```

```

    blast_velocity.y = obj->position.y - mouse_pos.y;

```



```

// distance between explosion and phy_obj or equivalently

// the magnitude of the vector obtained above

float distance = sqrtf(blast_velocity.x * blast_velocity.x + blast_velocity.y * bl

if (distance < 0.1f)

{

    distance = 0.1f;

}

if (distance - obj->radius <= radius)

{

    // normalization + scaling according to the radius

    blast_velocity.x = blast_velocity.x / distance * radius;

    blast_velocity.y = blast_velocity.y / distance * radius;

    obj->velocity = blast_velocity;

}

tmp = tmp->next;

}

for (int i = 0; i < 20; i++)

{

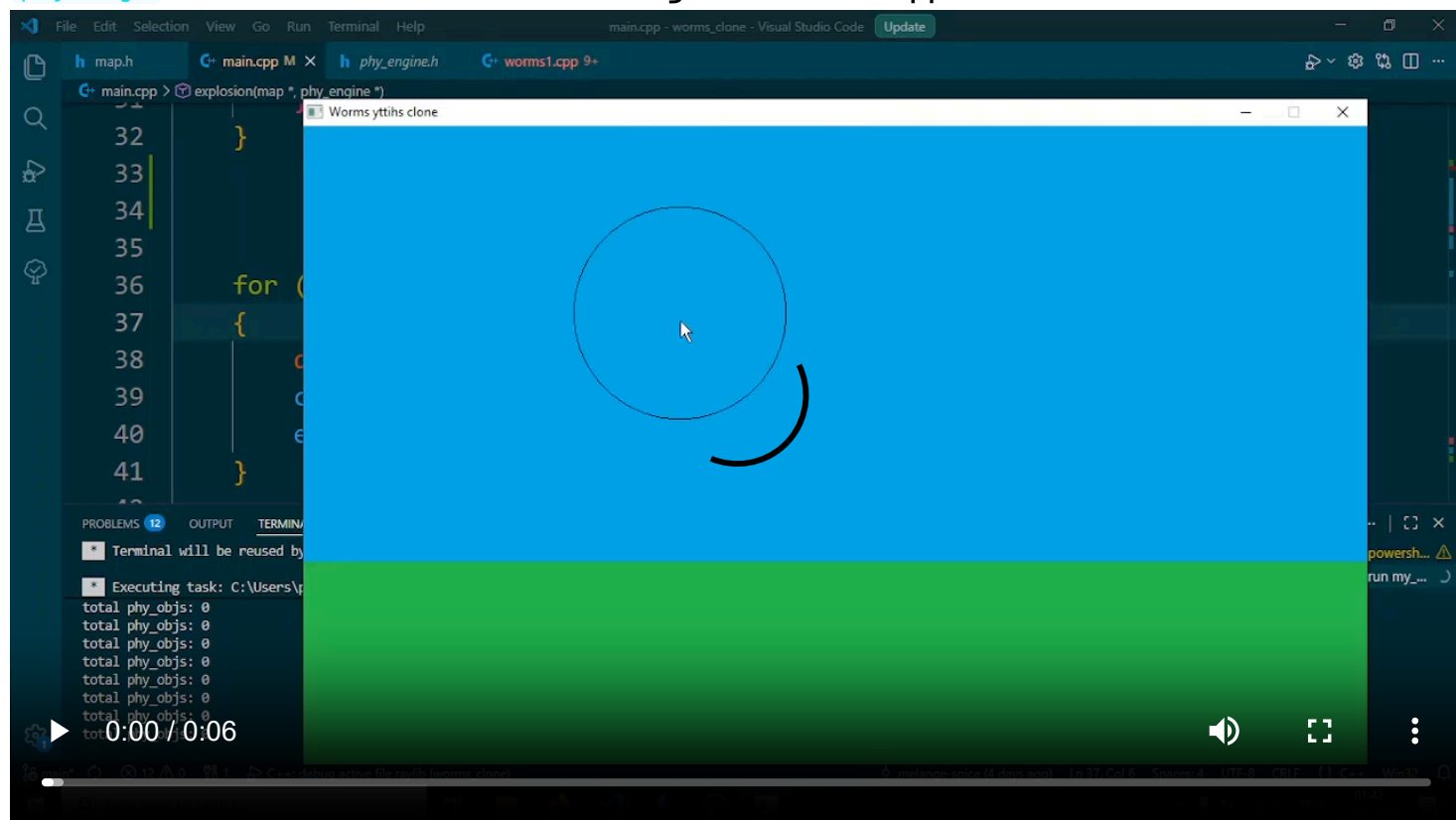
    debris *created_debris = nullptr;

    created_debris = new debris({mouse_pos.x + i, mouse_pos.y + i}, 6);

    engine->insertAtTail(created_debris);

```

Note how the debris is instantiated after applying shockwaves to all the `phy_objs`. This ensures the following does not happen:



sprintf

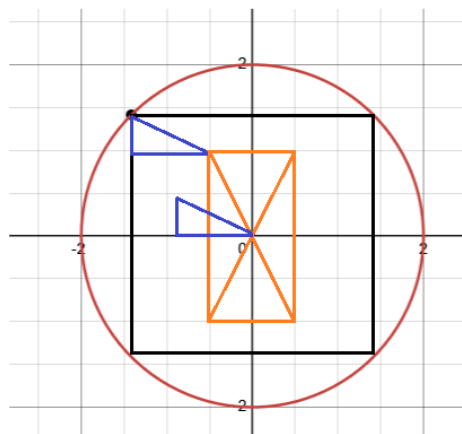
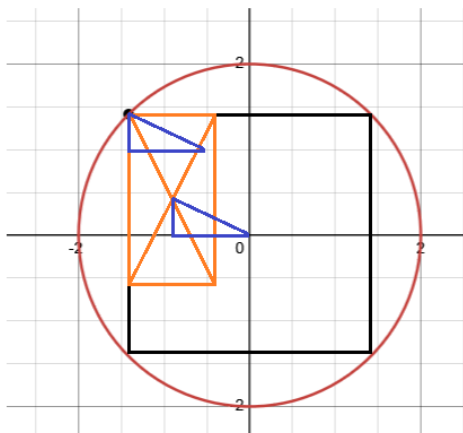
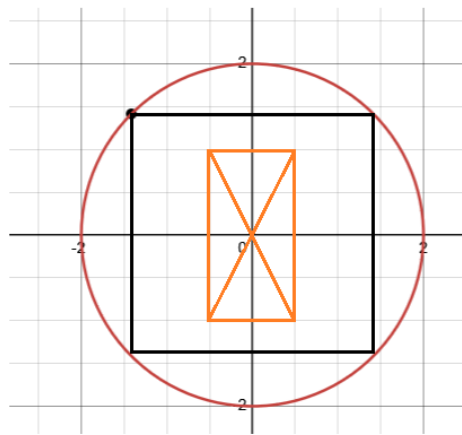
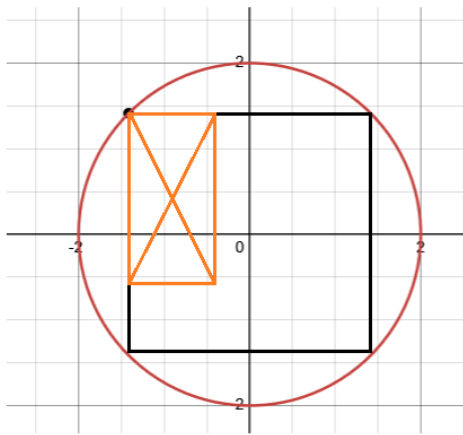
`sprintf` is specially useful to construct formatted strings:^[4]

```
// Example program to demonstrate sprintf()
#include <stdio.h>
int main()
{
    char buffer[50];
    int a = 10, b = 20, c;
    c = a + b;
    sprintf(buffer, "Sum of %d and %d is %d", a, b, c);
    snprintf(buffer, 50, "Sum of %d and %d is %d", a, b, c); //this is a better alternative to sprintf

    // The string "sum of 10 and 20 is 30" is stored
    // into buffer instead of printing on stdout
    printf("%s", buffer);

    return 0;
}
```

Sprite Centering



§

1. https://en.wikipedia.org/wiki/Equations_of_motion#Kinematic_quantities ↩
2. https://en.wikipedia.org/wiki/Equations_of_motion#Kinematic_quantities ↩
3. <https://youtu.be/EHlaJvQpW3U?t=1357> ↩
4. <https://www.geeksforgeeks.org/c/sprintf-in-c/> ↩